

Cryptography and Network Security Chapter 12

Fifth Edition
by William Stallings

Lecture slides by Lawrie Brown

Lecture slides by Lawrie Brown for “Cryptography and Network Security”, 5/e, by William Stallings, Chapter 12 – “Message Authentication Codes”.

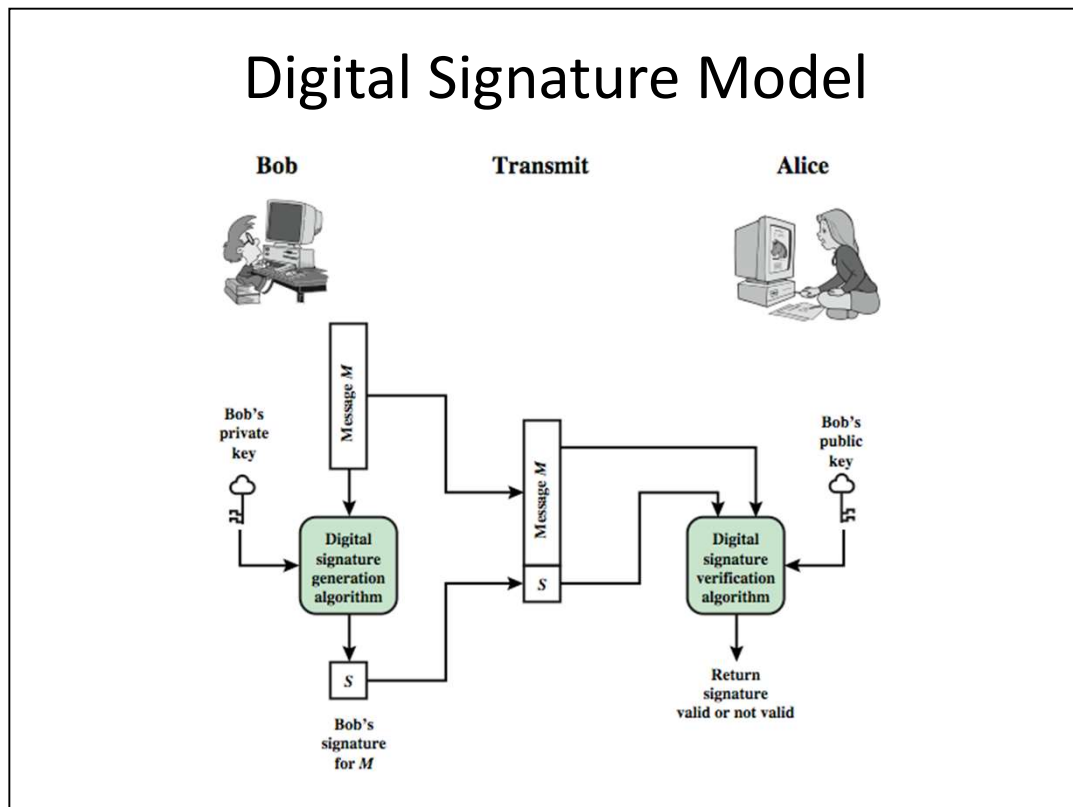
Digital Signatures

- have looked at message authentication
 - but does not address issues of lack of trust
- digital signatures provide the ability to:
 - verify author, date & time of signature
 - authenticate message contents
 - be verified by third parties to resolve disputes
- hence include authentication function with additional capabilities

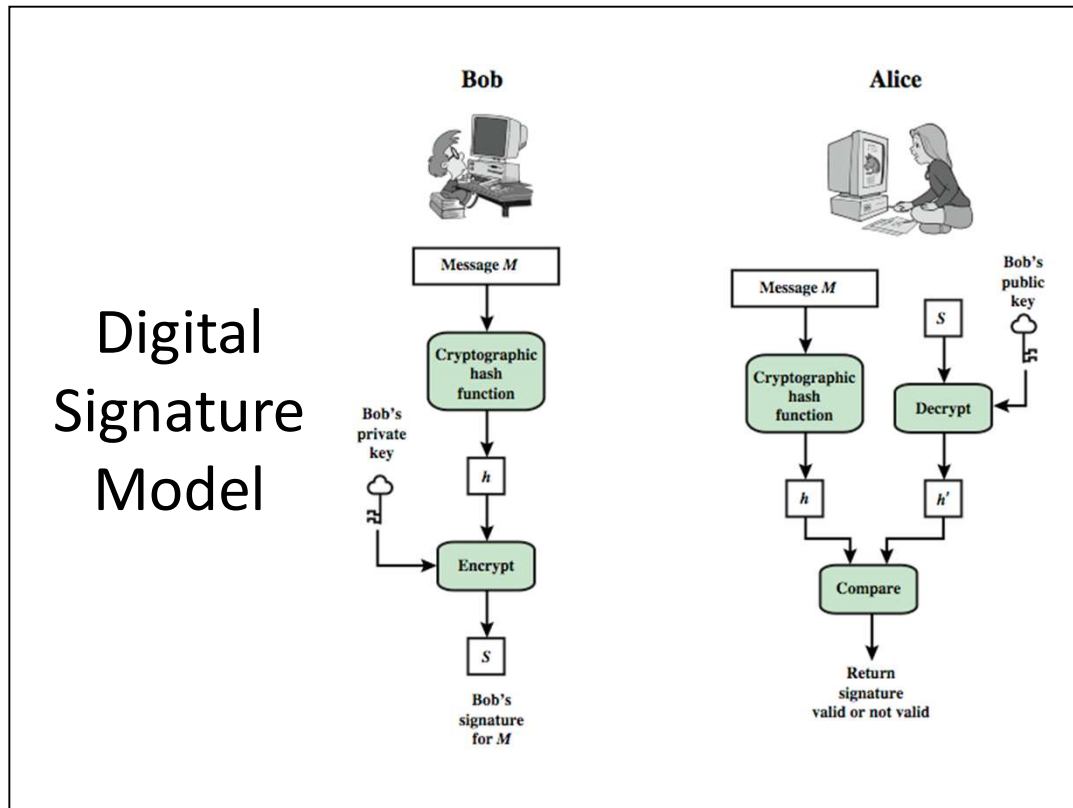
The most important development from the work on public-key cryptography is the digital signature. Message authentication protects two parties who exchange messages from any third party. However, it does not protect the two parties against each other either fraudulently creating, or denying creation, of a message. A digital signature is analogous to the handwritten signature, and provides a set of security capabilities that would be difficult to implement in any other way. It must have the following properties:

- It must verify the author and the date and time of the signature
- It must to authenticate the contents at the time of the signature
- It must be verifiable by third parties, to resolve disputes

Thus, the digital signature function includes the authentication function.



Stallings Figure 13.1 is a generic model of the process of making and using digital signatures. Bob can sign a message using a digital signature generation algorithm. The inputs to the algorithm are the message and Bob's private key. Any other user, say Alice, can verify the signature using a verification algorithm, whose inputs are the message, the signature, and Bob's public key.



In simplified terms, the essence of the digital signature mechanism is shown in Stallings Figure 13.2. This repeats the logic shown in Figure 11.3. On example, using RSA, is available at this book's Web site.

We begin this chapter with an overview of digital signatures. Then, we introduce the Digital Signature Standard (DSS).

Chapter 12 – Message Authentication Codes

- *At cats' green on the Sunday he took the message from the inside of the pillar and added Peter Moran's name to the two names already printed there in the "Brontosaur" code. The message now read: "Leviathan to Dragon: Martin Hillman, Trevor Allan, Peter Moran: observe and tail." What was the good of it John hardly knew. He felt better, he felt that at last he had made an attack on Peter Moran instead of waiting passively and effecting no retaliation. Besides, what was the use of being in possession of the key to the codes if he never took advantage of it?*
- —***Talking to Strange Men, Ruth Rendell***

Opening quote.

Message Authentication

- message authentication is concerned with:
 - protecting the integrity of a message
 - validating identity of originator
 - non-repudiation of origin (dispute resolution)
- will consider the security requirements
- then three alternative functions used:
 - hash function (see Ch 11)
 - message encryption
 - message authentication code (MAC)

One of the most fascinating and complex areas of cryptography is that of message authentication and the related area of digital signatures. We now consider how to protect message integrity (ie protection from modification), as well as confirming the identity of the sender. Generically this is the problem of message authentication, and in eCommerce applications is arguably more important than secrecy. Message Authentication is concerned with: protecting the integrity of a message, validating identity of originator, & non-repudiation of origin (dispute resolution). There are three types of functions that may be used to produce an authenticator: a hash function, message encryption, message authentication code (MAC). Hash functions, and how they may serve for message authentication, are discussed in Chapter 11. The remainder of this section briefly examines the remaining two topics. The remainder of the chapter elaborates on the topic of MACs.

Message Security Requirements

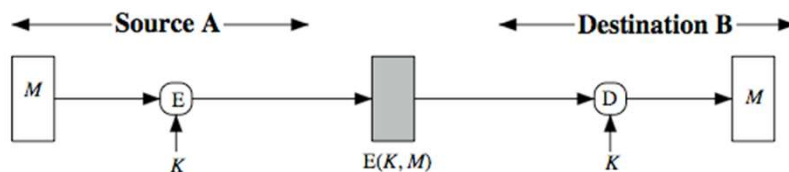
- **Disclosure:** Release of message contents
- **traffic analysis:** : Discovery of the pattern of traffic between parties
- **Masquerade:** Insertion of messages into the network
- **content modification**
- **sequence modification**
- **timing modification:** Delay or replay of messages
- **source repudiation:** Denial of transmission of message by source
- **destination repudiation:** Denial of receipt of message by destination

In the context of communications across a network, the attacks listed above can be identified, with more detail given in the text. The first two requirements (Disclosure: Release of message contents; and Traffic analysis: Discovery of the pattern of traffic between parties) belong in the realm of message confidentiality, and are handled using the encryption techniques already discussed. Measures to deal with items 3 through 6 (Masquerade: Insertion of messages into the network from a fraudulent source; Content modification: of the contents of a message; Sequence modification: to a sequence of messages between parties; and Timing modification: Delay or replay of messages) are generally regarded as message authentication. Mechanisms for dealing specifically with item 7 (Source repudiation: Denial of transmission of message by source) come under the heading of digital signatures. Generally, a digital signature technique will also counter some or all of the attacks listed under items 3 through 6. Dealing with item 8 (Destination repudiation: Denial of receipt of message by destination) may require a combination of the use of digital signatures and a protocol designed to counter this attack.

In summary, message authentication is a procedure to verify that received messages come from the alleged source and have not been altered. Message authentication may also verify sequencing and timeliness. A digital signature is an authentication technique that also includes measures to counter repudiation by the source.

Symmetric Message Encryption

- encryption can also provides authentication
- if symmetric encryption is used then:
 - receiver know sender must have created it
 - since only sender and receiver now key used
 - know content cannot of been altered
 - if message has suitable structure, redundancy or a checksum to detect any changes



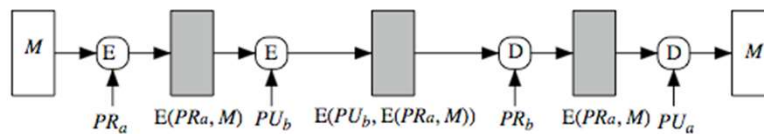
(a) Symmetric encryption: confidentiality and authentication

Message encryption by itself can provide a measure of authentication. The analysis differs for symmetric and public-key encryption schemes.

If use symmetric encryption, If no other party knows the key, then confidentiality is provided. As well, symmetric encryption provides authentication as well as confidentiality, since only the other party can have encrypted a properly constructed message (Stallings Figure 12.1a). Here, the ciphertext of the entire message serves as its authenticator, on the basis that only those who know the appropriate keys could have validly encrypted the message. This is provided you can recognize a valid message (ie if the message has suitable structure such as redundancy or a checksum to detect any changes).

Public-Key Message Encryption

- if public-key encryption is used:
 - encryption provides no confidence of sender
 - since anyone potentially knows public-key
 - however if
 - sender **signs** message using their private-key
 - then encrypts with recipients public key
 - have both secrecy and authentication
 - again need to recognize corrupted messages
 - but at cost of two public-key uses on message



(d) Public-key encryption: confidentiality, authentication, and signature

With public-key techniques, can use a digital signature which can only have been created by key owner to validate the integrity of the message contents. To provide both confidentiality and authentication, A can encrypt M first using its private key, which provides the digital signature, and then using B's public key, which provides confidentiality (Stallings Figure 12.1d). The disadvantage of this approach is that the public-key algorithm, which is complex, must be exercised four times rather than two in each communication.

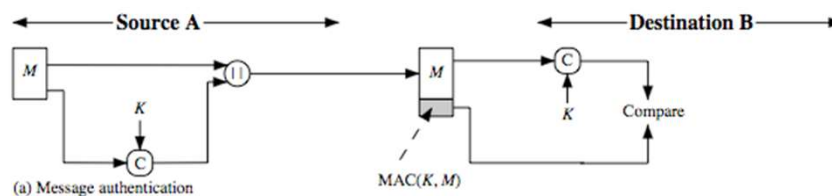
Message Authentication Code (MAC)

- generated by an algorithm that creates a small fixed-sized block
 - depending on both message and some key
 - like encryption though need not be reversible
- appended to message as a **signature**
- receiver performs same computation on message and checks it matches the MAC
- provides assurance that message is unaltered and comes from sender

An alternative authentication technique involves the use of a secret key to generate a small fixed-size block of data, known as a cryptographic checksum or MAC that is appended to the message. This technique assumes that two communicating parties, say A and B, share a common secret key K. A MAC function is similar to encryption, except that the MAC algorithm need not be reversible, as it must for decryption.

Message Authentication Code

- a small fixed-sized block of data
 - generated from message + secret key
 - $MAC = C(K, M)$
 - appended to message when sent



An alternative authentication technique involves the use of a secret key to generate a small fixed-size block of data, known as a cryptographic checksum or MAC that is appended to the message. This technique assumes that two communicating parties, say A and B, share a common secret key K . When A has a message to send to B, it calculates the MAC as a function of the message and the key: $MAC = C(K, M)$. The message plus MAC are transmitted to the intended recipient. The recipient performs the same calculation on the received message, using the same secret key, to generate a new MAC. The received MAC is compared to the calculated MAC (Stallings Figure 12.4a). If we assume that only the receiver and the sender know the identity of the secret key, and if the received MAC matches the calculated MAC, then the receiver is assured that the message has not been altered, is from the alleged sender, and if the message includes a sequence number then the receiver can be assured of the proper sequence because an attacker cannot successfully alter the sequence number. A MAC function is similar to encryption. One difference is that the MAC algorithm need not be reversible, as it must for decryption. In general, the MAC function is a many-to-one function.

Message Authentication Codes

- as shown the MAC provides authentication
- can also use encryption for secrecy
 - generally use separate keys for each
 - can compute MAC either before or after encryption
 - is generally regarded as better done before
- why use a MAC?
 - sometimes only authentication is needed
 - sometimes need authentication to persist longer than the encryption (eg. archival use)
- note that a MAC is **not** a digital signature

The process depicted on the previous slide provides authentication but not confidentiality, because the message as a whole is transmitted in the clear. Confidentiality can be provided by performing message encryption either after (see Stallings Figure 12.4b) or before (see Stallings Figure 12.4c) the MAC algorithm. In both these cases, two separate keys are needed, each of which is shared by the sender and the receiver. Typically, it is preferable to tie the authentication directly to the plaintext, so the method of Figure 12.4b is used. Can use MAC in circumstances where just authentication is needed (or needs to be kept), see text for examples (e.g. such as when the same message is broadcast to a number of destinations; when one side has a heavy load and cannot afford the time to decrypt all incoming messages; or do not need to keep messages secret, but must authenticate messages). Finally, note that the MAC does not provide a digital signature because both sender and receiver share the same key.

MAC Properties

- a MAC is a cryptographic checksum

$$\text{MAC} = C_K(M)$$

- condenses a variable-length message M
 - using a secret key K
 - to a fixed-sized authenticator
- is a many-to-one function
 - potentially many messages have same MAC
 - but finding these needs to be very difficult

A MAC (also known as a cryptographic checksum, fixed-length authenticator, or tag) is generated by a function C . The MAC is appended to the message at the source at a time when the message is assumed or known to be correct. The receiver authenticates that message by re-computing the MAC.

The MAC function is a many-to-one function, since potentially many arbitrarily long messages can be condensed to the same summary value, but don't want finding them to be easy (see text for discussion)!

Requirements for MACs

- taking into account the types of attacks
- need the MAC to satisfy the following:
 1. knowing a message and MAC, is infeasible to find another message with same MAC
 2. MACs should be uniformly distributed
 3. MAC should depend equally on all bits of the message

In assessing the security of a MAC function, we need to consider the types of attacks that may be mounted against it. Hence it needs to satisfy the listed requirements.

The first requirement deals with message replacement attacks, in which an opponent is able to construct a new message to match a given MAC, even though the opponent does not know and does not learn the key.

The second requirement deals with the need to thwart a brute-force attack based on chosen plaintext.

The final requirement dictates that the authentication algorithm should not be weaker with respect to certain parts or bits of the message than others.

Security of MACs

- like block ciphers have:
- **brute-force** attacks exploiting
 - strong collision resistance hash have cost $2^{m/2}$
 - 128-bit hash looks vulnerable, 160-bits better
 - MACs with known message-MAC pairs
 - can either attack key space (cf key search) or MAC
 - at least 128-bit MAC is needed for security

Just as with symmetric and public-key encryption, we can group attacks on hash functions and MACs into two categories: brute-force attacks and cryptanalysis.

A brute-force attack on a MAC is a more difficult undertaking than a brute-force attack on a hash function because it requires known message-tag pairs. The strength of a hash function against brute-force attacks depends solely on the length of the hash code produced by the algorithm, with cost $O(2^{m/2})$. A brute-force attack on a MAC has cost related to $\min(2^k, 2^n)$, similar to symmetric encryption algorithms. It would appear reasonable to require that the key length and MAC length satisfy a relationship such as $\min(k, n) \geq N$, where N is perhaps in the range of 128 bits.

Security of MACs

- **cryptanalytic attacks** exploit structure
 - like block ciphers want brute-force attacks to be the best alternative
- more variety of MACs so harder to generalize about cryptanalysis

As with encryption algorithms, cryptanalytic attacks on hash functions and MAC algorithms seek to exploit some property of the algorithm to perform some attack other than an exhaustive search. The way to measure the resistance of a hash or MAC algorithm to cryptanalysis is to compare its strength to the effort required for a brute-force attack. That is, an ideal hash or MAC algorithm will require a cryptanalytic effort greater than or equal to the brute-force effort. There is much more variety in the structure of MACs than in hash functions, so it is difficult to generalize about the cryptanalysis of MACs. Further, far less work has been done on developing such attacks.

Keyed Hash Functions as MACs

- want a MAC based on a hash function
 - because hash functions are generally faster
 - crypto hash function code is widely available
- hash includes a key along with message
- original proposal:
$$\text{KeyedHash} = \text{Hash}(\text{Key} \parallel \text{Message})$$
 - some weaknesses were found with this
- eventually led to development of HMAC

In recent years, there has been increased interest in developing a MAC derived from a cryptographic hash function, because they generally execute faster in software than symmetric block ciphers, and because code for cryptographic hash functions is widely available. A hash function such as SHA was not designed for use as a MAC and cannot be used directly for that purpose because it does not rely on a secret key. There have been a number of proposals for the incorporation of a secret key into an existing hash algorithm, originally by just prepending a key to the message. Problems were found with these earlier, simpler proposals, but they resulted in the development of HMAC.